

Building a Serverless Data Processing Pipeline on AWS

Course	MSc in Data Science and AI: Cloud and DevOps Foundations
Name	Shaik Khaja Nayab Rasool
Roll Number	2025em1100102

Preface

A serverless, event-driven pipeline on AWS that automatically processes CSV files upon upload. Files stored in Amazon S3 trigger an AWS Lambda function to compute key metrics, which are logged in Amazon CloudWatch and stored in Amazon DynamoDB. The solution runs without manual intervention, ensuring real-time processing & efficient resource usage. Security is managed through IAM, while design focuses on scalability, cost-efficiency & seamless integration across services.

Part A: IAM Setup

a. User & Access Configuration

An IAM user named **Assignment** was created to perform all activities part of assignment, instead of using root account. Console access was enabled for user and all subsequent tasks were executed using non-root account.

Assignment Info

Summary
ARN
 arn:aws:iam::338183195533:user/Assignment

Created
March 19, 2026, 12:21 (UTC+05:30)

Console access
 Enabled without MFA

Last console sign-in
 Today

To enforce structured access control, user was added to an IAM group named **AssignmentGroup**, instead of attaching permissions directly to align with AWS best practices by enabling centralized permission management & improving scalability.

AssignmentGroup Info

Summary
User group name
AssignmentGroup

Creation time
March 19, 2026, 12:21 (UTC+05:30)

ARN
 arn:aws:iam::338183195533:group/AssignmentGroup

b. Permissions & Policy Design

AWS managed policies were attached to **AssignmentGroup**, to provide **service-level access**. Instead of assigning account-wide administrative privileges, access was restricted to specific AWS services required for the assignment for them to interact properly, following the principle of least privilege while still allowing pipeline to work.

Permissions policies (5)
Permissions are defined by policies attached to the user directly or through groups.

Filter by Type
All types

Policy name	Type	Attached via
  CloudWatchReadOnlyAccess	AWS managed	Group AssignmentGroup
  DynamoDBLeastPrivilegePolicy	Customer managed	Group AssignmentGroup
  IAMLeastPrivilegePolicy	Customer managed	Group AssignmentGroup
  LambdaLeastPrivilegePolicy	Customer managed	Group AssignmentGroup
  S3LeastPrivilegePolicy	Customer managed	Group AssignmentGroup

Service	Policy	Purpose
S3	S3LeastPrivilegePolicy	• PutObject • GetObject • ListBucket
Lambda	LambdaLeastPrivilegePolicy	• CreateFunction • UpdateFunctionCode • InvokeFunction • AddPermission
DynamoDB	DynamoDBLeastPrivilegePolicy	• PutItem • GetItem • DescribeTable
IAM	IAMLeastPrivilegePolicy	• CreateRole • AttachRolePolicy • PassRole • GetRole
CloudWatch	CloudWatchReadOnlyAccess	Viewlogs(GetLogEvents, DescribeLogStreams)

c. Security Best Practices

- **Avoided Root Usage:** Root account was used only for initial setup, all subsequent actions were performed using the IAM user to minimize security risks.
- **Group-Based Access Control:** Permissions were assigned via IAM groups rather than directly to users, ensuring better manageability and adherence to role-based access principles. Group-based permission management simplifies access control in multi-user environments
- **Role-Based Access for Lambda Execution:** A dedicated IAM execution role is required for Lambda function to interact with AWS services at runtime, granted minimal permissions to read objects from S3, write logs to CloudWatch & insert records into DynamoDB. IAM user requires iam:PassRole permission to associate execution role with Lambda function, such separation ensures secure & controlled access during execution.
- **Service-Scoped Permissions:** Broad administrative access (e.g., AdministratorAccess) was intentionally avoided, permissions limited to only required AWS services. The IAM setup ensures balance between listed facets, to ensure users granted only access necessary to perform their tasks. Over-permissioning (e.g., full administrative access) increases security risks
 - **Security:** Avoids root access & limiting permissions scope.
 - **Usability:** Enables sufficient access, without operational blockers.
 - **Scalability:** By using group-based permission management.

PART B: Architecture Overview

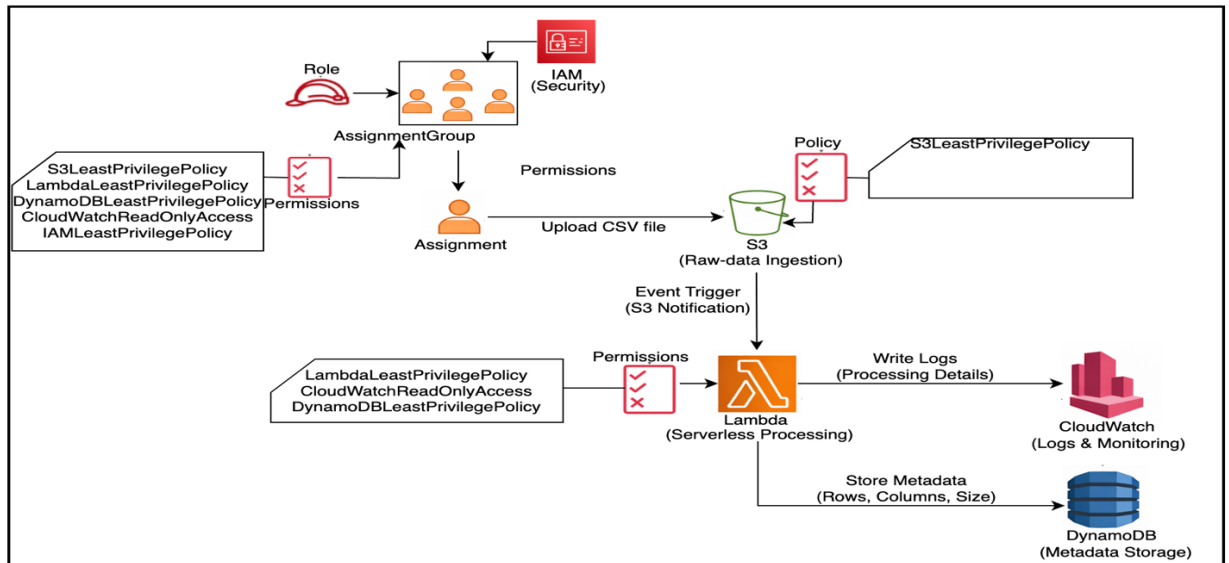
Technical Workflow

1. A user uploads CSV file into an S3 bucket, which acts as data ingestion layer.
2. Upon file upload, an event notification automatically triggers Lambda function.
3. Lambda function reads & processes file, computes file-level metrics such as rows count, columns count, file size etc. & logs execution details to CloudWatch for monitoring.
4. Processed metadata is stored in an DynamoDB table for persistence & future analysis.

5. **Lambda** function operates using **dedicated IAM** execution role that provides secure & least-privilege access to resources.

Overview

The architecture follows a serverless, event-driven design using AWS managed services.



Why This Architecture is used ?

The architecture is serverless & event-driven, eliminating need for manual intervention or infrastructure management, highly scalable as Lambda automatically scales based on incoming events. The design is cost-effective as resources used only when triggered. Additionally, security enforced through IAM roles & observability is achieved via CloudWatch logs & DynamoDB records. Overall, architecture proposal is to build a scalable, cost-effective & secure data processing pipeline. The implementation reflects real-world cloud design principles by integrating multiple services while ensuring observability & operational efficiency.

Trade-Offs

- Reduced control due to managed services abstraction.
- Cold start latency in Lambda executions.
- Increased complexity in debugging distributed workflows.
- Service limits may constrain large-scale processing.

How services interact ?

The user uploads a CSV file into an S3 bucket, which acts as data ingestion layer. The upload event triggers an Lambda function automatically, which processes file, computes metrics such as row & column count, and stores results in an Amazon DynamoDB table. Simultaneously, execution details are logged in CloudWatch for monitoring.

How security is enforced ?

Security is enforced using IAM. A non-root IAM user with restricted permissions is used for resource creation. Additionally, Lambda function operates with dedicated execution role that grants only

required permissions to access S3, write logs to CloudWatch & insert records into DynamoDB, following principle of least privilege.

Why event-driven design is used ?

The architecture is event-driven because Lambda function is automatically triggered by S3 upload events eliminating need for manual intervention or continuous polling, making system efficient, scalable & cost-effective for real-time data processing.

Part C: AWS Resource Setup

a. S3 | Bucket: assignment-csv-processing-2025em1100102

Serves as data ingestion layer for CSV file(s), each upload event automatically triggers Lambda function through event-driven workflow and ensures every incoming file gets processed in real time making it a central entry point for initiating pipeline. Output validation screenshots are enclosed.

Upload: status

After you navigate away from this page, the following information is no longer available.

Summary

Destination

s3://assignment-csv-processing-2025em1100102

Succeeded

1 file, 2.8 KB (100.00%)

Failed

0 files, 0 B (0%)

assignment-csv-processing-2025em1100102

Info

Objects

Metadata

Properties

Permissions

Metrics

Management

Access Points

Objects (4)

Copy S3 URI

Copy URL

Download

Open

Delete

Actions

Create

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 inventory](#) to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions.

Find objects by prefix

	Name	Type	Last modified	Size	Storage class
☐	seller_catalog.csv	csv	March 20, 2026, 02:00:56 (UTC+05:30)	2.8 KB	Standard
☐	seller_catalog1.csv	csv	March 20, 2026, 02:06:37 (UTC+05:30)	2.8 KB	Standard
☐	seller_catalog2.csv	csv	March 20, 2026, 02:06:59 (UTC+05:30)	2.8 KB	Standard
☐	seller_catalog3.csv	csv	March 20, 2026, 02:07:19 (UTC+05:30)	2.8 KB	Standard

b. Lambda Configuration | Function: file-processing-lambda--2025em1100102

Acts as processing engine of pipeline, automatically triggered by S3 events to process uploaded files for reading CSV data, computing key metrics to forwards results to CloudWatch & DynamoDB for monitoring & storage. Baseline Lambda script provided in assignment was used without modification.

Function overview

Diagram

Template

file-processing-lambda--2025em1100102

Layers

(0)

S3

+ Add trigger

+ Add destination

Description

Last modified

4 minutes ago

Function ARN

arn:aws:lambda:us-east-1:338183195533:function:file-processing-lambda--2025em1100102

Function URL

Info

Trigger

S3: assignment-csv-processing-2025em1100102
arn:aws:s3:::assignment-csv-processing-2025em1100102

Details

Bucket arn: arn:aws:s3:::assignment-csv-processing-2025em1100102
Event types: s3:ObjectCreated:Put
isComplexStatement: No
Notification name: a178a8ef-2f97-4d16-a83e-649e234fffbcb
Service principal: s3.amazonaws.com
Source account: 338183195533
Statement ID: lambda-c84fba8a-dab4-4902-9429-d8cf8a2994dc

assignment-csv-processing-2025em1100102 Info

Objects (5)

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 Inventory](#) to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. [Learn more](#)

Find objects by prefix

Name	Type	Last modified	Size	Storage class
seller_catalog.csv	csv	March 20, 2026, 02:00:56 (UTC+05:30)	2.8 KB	Standard
seller_catalog1.csv	csv	March 20, 2026, 02:06:57 (UTC+05:30)	2.8 KB	Standard
seller_catalog2.csv	csv	March 20, 2026, 02:06:59 (UTC+05:30)	2.8 KB	Standard
seller_catalog3.csv	csv	March 20, 2026, 02:07:19 (UTC+05:30)	2.8 KB	Standard
seller_catalog4.csv	csv	April 12, 2026, 11:57:05 (UTC+05:30)	2.8 KB	Standard

c. CloudWatch Logs | Log group: /aws/lambda/file-processing-lambda--2025em1100102

CloudWatch is used to monitor & validate execution of Lambda function, captures logs containing processed file details such as file name, number of rows & columns, enabling effective debugging & confirming correct pipeline execution. Output validation screenshots are enclosed.

CloudWatch > Log management > /aws/lambda/file-processing-lambda--2025em1100102 > 2026/03/19/[LATEST]Bb25131e6b5945f3af3abe093e97bb29

CloudWatch

Log events

You can use the filter bar below to search for and match terms, phrases, or values in your log events. [Learn more about filter patterns](#)

Filter events - press enter to search

Clear 1m 30m 1h 12h Custom UTC timezone

Timestamp	Message
2026-03-19T20:30:56.729Z	INIT_START Runtime Version: python:3.10.mainlinev2.v3 Runtime Version ARN: arn:aws:lambda:us-east-1::runtime:a83ff4af723a0d44f64166b2291af6cad56a30ba3ab5e17532c29ecf9f5b3e0
2026-03-19T20:30:57.348Z	START RequestId: 20f2f350-5fe2-483d-931c-29e813539e2b Version: \$LATEST
2026-03-19T20:30:57.642Z	Processed File Metrics: {"file_name": "seller_catalog.csv", "file_type": ".csv", "file_size_bytes": 2894, "rows": 49, "columns": 6, "processed_at": "2026-03-19T20:30:57.348913"}
2026-03-19T20:30:57.839Z	END RequestId: 20f2f350-5fe2-483d-931c-29e813539e2b
2026-03-19T20:30:57.839Z	REPORT RequestId: 20f2f350-5fe2-483d-931c-29e813539e2b Duration: 490.84 ms Billed Duration: 1107 ms Memory Size: 128 MB Max Memory Used: 91 MB Init Duration: 615.34 ms

d. DynamoDB Configuration | Table: FileProcessingMetadata

DynamoDB is used to store processed file metadata generated by Lambda function, maintains structured records such as file name, size, row count & column count providing a persistent & queryable store to validate & track processed files. Output validation screenshots are enclosed.

DynamoDB > Tables

DynamoDB

Tables (1)

Find tables

Filter by tag Any tag key

Filter by tag value Any tag value

Name	Status	Partition key	Sort key	Indexes	Replication Regions	Deletion protection	Favorite	Read capacity mode	Write capacity mode	Total size	Table class
FileProcessingMetadata	Active	file_name (S)	-	0	0	Off	☆	On-demand	On-demand	0 bytes	Standard

Table: FileProcessingMetadata - Items returned (0)

Scan started on March 20, 2026, 01:38:17

file_name (String)

No items

No items to display.

Create item

Table: FileProcessingMetadata - Items returned (1)

Scan started on March 20, 2026, 02:01:42

file_name (String)	columns	file_size_bytes	file_type	processed_at	rows
seller_catalog.csv	6	2894	.csv	2026-03-19T20:30:57.348913	49

Table: FileProcessingMetadata - Items returned (5)

Scan started on April 12, 2026, 13:15:04

file_name (String)	columns	file_size_bytes	file_type	processed_at	rows
seller_catalog.csv	6	2894	.csv	2026-03-19T20:30:57.348913	49
seller_catalog1.csv	6	2894	.csv	2026-03-19T20:36:58.385082	49
seller_catalog2.csv	6	2894	.csv	2026-03-19T20:36:59.685514	49
seller_catalog3.csv	6	2894	.csv	2026-03-19T20:37:19.485611	49
seller_catalog4.csv	6	2894	.csv	2026-04-12T06:27:05.063473	49